

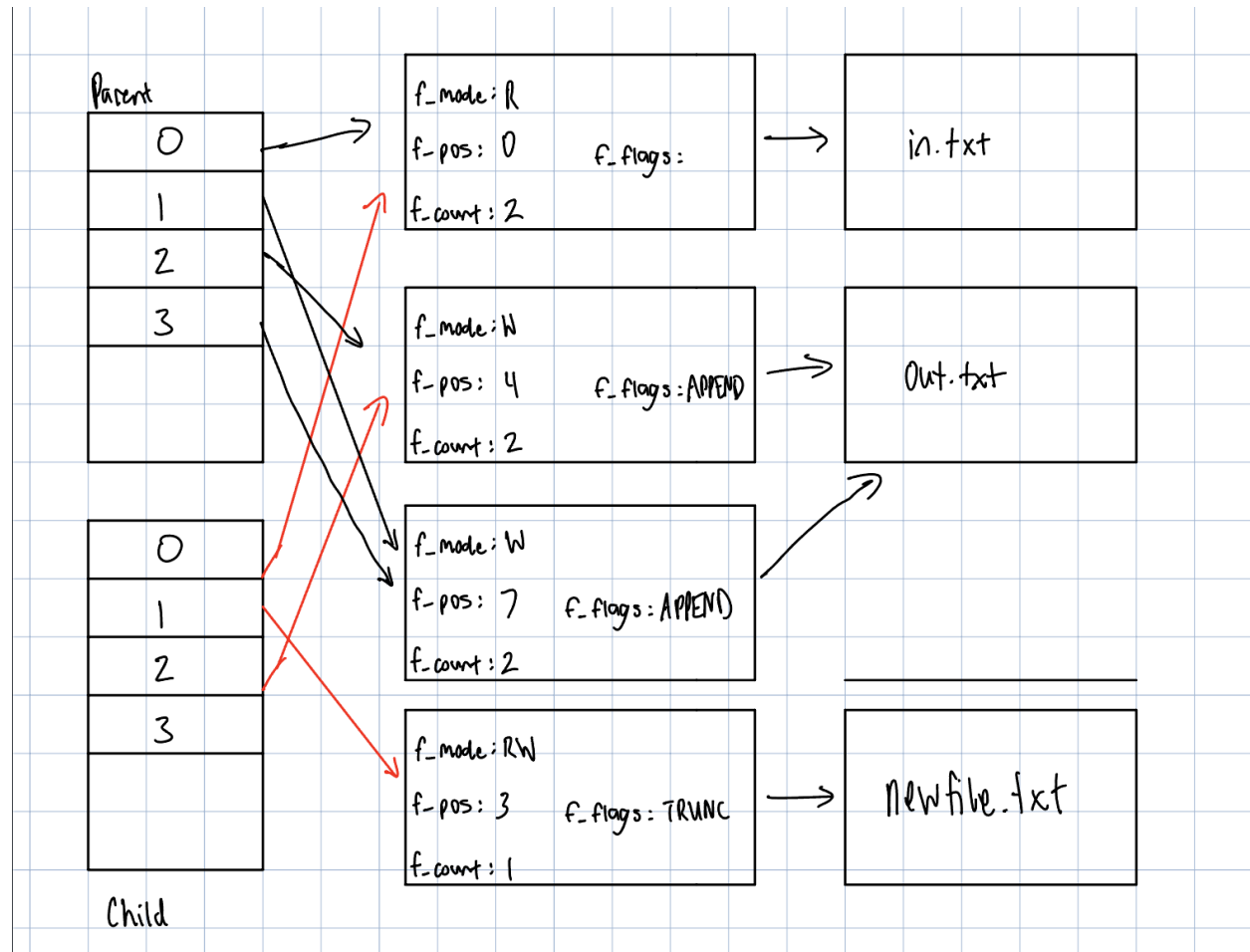
1a. The `/bin/bash` interpreter runs inside process 1000: its list of arguments are as followed. Argument 0 is the name of the program itself (`/bin/bash`), argument one is the flag `-vx` (for verbose and `xtrace`). Argument 2-4 involves what happens to the following shell: argument 2 is `"script.sh"`, and its arguments (the `*n.c` files) are also expanded. This means arguments 3 and 4 are `broken.c` and `main.c` respectively.

1b. The PID of child processes does not match the PID of the parent process and each process gets its own register values. This is because if they shared the same register values, then the program counter would also be shared which means tasking would become troublesome/interfering.

1c. Global variables are initialized to zeroes since they are declared by the BSS which fills the space with zeroes; as chars these are interpreted as the null terminator, which effectively means the string immediately null terminates, so that we always expect the print output to be **XY**.

1d. It will print Message 1 and then seg fault. This is because the terminal as `stdout` (given no IO redirection) flushes on newlines. Since Message 1 ends in a newline, it flushes, but Message 2 specifically does not include this newline. This means that instead of flushing (outputting to the screen), the next instruction is `ran` which is an obvious seg fault, causing the program to terminate there.

2. (next page)



2.

